technically we are disconnecting the observer, and then setting up a callback with Promise.resolve() that calls start to reset the observer only after the sortTargets call is complete. Otherwise the observer might restart while the sort is happening and you'd still get the infinite loop.

The mutation change records specify the type of the change (either attributes, characterData, or childList) and the target, which is the actual DOM element being mutated. The record then specifies other information about the change itself in attributes named addedNodes, removedNodes, and attributeName. (A full list of mutation types can be found online.[70])

Possible options, and therefore mutation types, are as follows (all values default to false):

- attributes: If true, the MutationObserver triggers the callback if the value of an attribute on the element changes. If the option attributeOldValue is also true, the previous value of the attribute is passed to the callback at mutationRecord.oldValue. If attributeFilter is passed as an option, it's a string list of attribute names that are the only attributes whose changes we care about.

- characterData: If true, the MutationObserver triggers the callback if the text inside the element changes. If characterDataOldValue is also true, the old value is also added to the MutationRecord at mutationRecord.oldValue.

- childlist: If true, the MutationObserver triggers the callback if a new child is added to the target element or an existing child is removed.

- subtree: If true, the MutationObserver also looks at changes recursively down the DOM tree of the target element; otherwise, only changes to the target element apply.

In many cases, your callback function would switch on the type of the change or take advantage of the mutation record data, but in our case, we don't really need that information. All we care about here is that the childList has changed.